IFPB

Introdução ao **Desenvolvimento** de **Sistemas Web** com **Django**

Ministrante: **Marcos Santos**

Linguagem **Python**

Python é uma linguagem de programação altamente recomendada para desenvolvedores iniciantes devido sua facilidade com a sintaxe, mas isso não diminui sua potência, pelo contrário com menos burocracia mais flexibilidade o desenvolvedor tem para conseguir a resolução de seu problema.

```python
nome = "João"

idade = 23

casado = False


def ShowInfo():

    print(nome + str(idade) + str(casado))


ShowInfo() #Executa a função
```

IFPB

Arrays **Python**

```
Lista = ["Banana", "Maçã", "Uva", "Pêra", "Abacaxi"]

print(Lista) #Resultado: "Banana, Maçã, Uva, Pêra, Abacaxi"


ListaOrdenada = sorted(Lista)

print(ListaOrdenada) #Resultado: ['Abacaxi', 'Banana', 'Maçã', 'Pêra', 'Uva']
```

IFPB

Logica **Python**

```
nome = input("Diga seu nome: ")

idade = int(input("Diga sua idade: "))



if(idade >= 18):

    print(nome + " é maior de idade!")

else:

    print(nome + " é menor de idade!")
```

IFPB

Framework **Django**

O Django é um framework web Python de código aberto, que se destaca por oferecer um ambiente que simplifica a criação de soluções web escaláveis, ao mesmo tempo, em que promove o desenvolvimento rápido e um design limpo, proporcionando ferramentas robustas e eficientes para pessoas desenvolvedoras.

fonte: www.alura.com.br/artigos/django-framework

IFPB

Por que usar **Django?**

🔋 Batteries included

Admin, autenticação, ORM, formulários e muito mais — tudo embutido, sem precisar instalar pacotes extras.

🔒 Seguro por padrão

Proteção contra CSRF, XSS e SQL Injection já vêm configuradas, seguindo boas práticas automaticamente.

📈 Escalável e confiável

Usado por Instagram, Spotify e Pinterest — funciona tanto em pequenos projetos quanto em grandes sistemas.

🐍 Python puro

Tudo escrito em Python: lógica, banco de dados, templates. Uma linguagem, um ecossistema.

fonte: www.alura.com.br/artigos/django-framework

Como funciona **a Web?**

fonte: www.alura.com.br/artigos/django-framework

Arquitetura MVT/MVC

O padrão MVT divide o desenvolvimento web em três componentes principais: Model (Modelo), View (Visão) e Template (Template).

fonte: www.alura.com.br/artigos/django-framework

IFPB

MVT — **Diagrama**

M

Model

Representa os dados
e interage com o
banco de dados via ORM

→

V

View

Recebe a requisição,
consulta o Model
e retorna a resposta

→

T

Template

Arquivo HTML com
tags Django para
exibir os dados

fonte: www.alura.com.br/artigos/django-framework

IFPB

Model

```
from django.db import models

class Livro(models.Model):

    nome = models.CharField(max_length=100)

    preco = models.DecimalField(max_digits=8, decimal_places=2)

    descricao = models.TextField()

    def __str__(self):

        return self.nome
```

fonte: www.alura.com.br/artigos/django-framework

IFPB

Model **ORM**

O Django utiliza o ORM (Object-Relational Mapping), que permite que você utilize classes e objetos no Python para mapear as tabelas do banco de dados, fazendo a criação, consulta, atualização e exclusão de dados com mais segurança e facilidade.

fonte: www.alura.com.br/artigos/django-framework

IFPB

ORM na **prática**

SQL vs Django ORM — mesma operação, formas diferentes:

SQL puro

```
SELECT * FROM livros

WHERE preco < 50

ORDER BY nome ASC;
```

Django ORM

```
Livro.objects.filter(

    preco__lt=50

).order_by("nome")
```

✅ **Sem SQL, sem erros de sintaxe, com segurança contra SQL Injection automática!**

View

É responsável por processar as solicitações HTTP do usuário e retornar as respostas desse processamento. As views são funções e classes que lidam com a lógica de negócios da aplicação, sendo projetadas para serem reutilizáveis e utilizadas em diferentes aplicações de um mesmo projeto.

fonte: www.alura.com.br/artigos/django-framework

IFPB

View

```
from django.shortcuts import render

from .models import Livro

def lista_de_livros(request):

    livros = Livro.objects.all()

    return render(request, 'loja/lista_de_livros.html', {'livros': livros})
```

fonte: www.alura.com.br/artigos/django-framework

IFPB

Template

Como o próprio nome indica, é responsável pela apresentação e pela renderização dos dados para o usuário. Eles são arquivos que contêm um código HTML com marcações especiais do Django, que permitem que você insira dinamicamente os dados dos modelos e crie páginas dinâmicas e interativas.

fonte: www.alura.com.br/artigos/django-framework

IFPB

Template

```html
<!DOCTYPE html>
<html>
<head>
    <title>Lista de Livros</title>
</head>
<body>
    <h1>Lista de Livros</h1>
    <ul>
        {% for livro in livros %}
            <li>
                <h2>{{ livros.nome }}</h2>
                <p>Preço: R$ {{ livros.preco }}</p>
                <p>{{ livros.descricao }}</p>
            </li>
        {% endfor %}
    </ul>
</body>
```

snappify.com

IFPB

URLs e **Roteamento**

```
from django.urls import path

from . import views



urlpatterns = [

    path('',          views.lista,   name='lista'),

    path('<int:pk>/', views.detalhe, name='detalhe'),

    path('novo/',     views.criar,   name='criar'),

]
```

fonte: www.alura.com.br/artigos/django-framework

IFPB

Django **Admin**

Um painel administrativo completo gerado automaticamente pelo Django — sem escrever nenhum código extra de interface.

```
# admin.py

from django.contrib import admin

from .models import Livro



@admin.register(Livro)

class LivroAdmin(admin.ModelAdmin):

    list_display  = ('nome', 'preco')

    search_fields = ('nome',)
```

Acesse http://127.0.0.1:8000/admin/ e crie um superusuário com python manage.py createsuperuser

IFPB

Ambiente de **Desenvolvimento**

1. **Instalar Python 3**

```
$ python --version
```

2. **Instalar o Django**

```
$ pip install django
```

3. **Criar o projeto**

```
$ django-admin startproject meu_projeto
```

4. **Criar a aplicação**

```
$ python manage.py startapp minha_app
```

5. **Rodar o servidor**

```
$ python manage.py runserver
```

✅ Após o passo 5 acesse: http://127.0.0.1:8000

IFPB

🚀 Mãos na Massa!

Agora vamos construir juntos: 🎭 Mural de Memes

🗄️ Model

Estrutura do banco de dados

⚙️ Views

Lógica e processamento

🌐 Templates

Interface com Bootstrap 5

📋 Forms

Envio de novos memes

🔧 Admin

Gerenciamento de dados

⏱️ *Duração: 35–45 minutos*